

Tutor Adaptativo de Fundamentos de Programación

Informe del Proyecto

Fundamentos de Inteligencia Artificial
Summer Camp 2026 — CyberMinds EPN

Autor: César Oswaldo Yunga Nieves

Modalidad: Individual

Repositorio: <https://github.com/Kaiman-p/tutor-adaptativo-ia>

Fecha: Agosto 2026

Índice

1. Definición del Problema	2
1.1. Contexto	2
1.2. Impacto	2
1.3. La solución que propongo	2
2. Qué hace la IA en este proyecto	3
3. Arquitectura de Implementación	3
3.1. Cómo fluyen los datos	3
3.2. Los cuatro lenguajes y sus referencias	4
3.3. Componentes técnicos	5
3.4. Herramientas y tecnologías utilizadas	5
4. Sistema de Aprendizaje Progresivo	5
4.1. Ayuda 1: Lección completa (contenido estático)	6
4.2. Ayuda 2: Pista paso a paso (IA en tiempo real)	6
4.3. Ayuda 3: Solución paso a paso (solucionario detallado)	7
4.4. Re-explicación adaptativa (IA en tiempo real)	7
5. Evidencia de Ejecución	8
5.1. El programa en ejecución	8
5.2. Verificación de los 4 lenguajes con análisis real de IA	9
5.3. El prompt de sistema (núcleo de IA)	11
6. Uso de la IA como Copiloto (Trazabilidad)	11
6.1. Distinción importante	11
6.2. Herramienta de copiloto utilizada	12
6.3. Modelos usados en tiempo de ejecución	12
6.4. Nota sobre depuración de proveedores	12
7. Cómo Ejecutar el Proyecto	12
7.1. Requisitos	12
7.2. Instalación y ejecución	13
8. Resultados y Verificación	13
9. Conclusiones	13

1. Definición del Problema

1.1. Contexto

Cualquiera que haya empezado a aprender a programar conoce esta sensación: escribes tu código, le das a “ejecutar”, y lo único que recibes de vuelta es un “correcto” o un “incorrecto”. Nada más. Si falló, no sabes por qué; y si funcionó, tampoco sabes si de verdad entendiste lo que hiciste o simplemente tuviste suerte copiando la estructura de un ejemplo anterior.

Ese vacío es un problema real. Los libros de referencia de cada lenguaje insisten, cada uno a su manera, en que aprender a programar no se trata solo de que el programa “corra”, sino de comprender los mecanismos que hay debajo. Kernighan y Ritchie, en su clásico sobre C, son famosos por enfocarse en que el estudiante entienda de verdad cómo funcionan los punteros y la memoria, no solo en que el código compile [1]. Esa misma idea — comprender, no memorizar — es la que motiva este proyecto.

Hay además una razón más personal, y más actual, detrás de este proyecto. Hoy en día es facilísimo pedirle a una IA que escriba código, y que lo escriba bien. Pero eso mismo esconde un riesgo: si no tienes los fundamentos, no puedes saber si lo que la IA generó hace lo que de verdad necesitas, no puedes detectar un error sutil, ni puedes adaptar esa solución a un caso distinto. Depender de la IA sin entender lo que produce es quedar a ciegas de tu propio código. Por eso quise construir una herramienta que, usando la IA como motor, me obligara a entender los fundamentos por mi propia cuenta — no a saltármelos.

El problema, entonces, es que las herramientas típicas de práctica te dicen *si* acertaste, pero no *qué tan bien* entendiste el concepto que estabas practicando. Y sin esa información, es fácil caer en una práctica poco eficiente.

1.2. Impacto

¿Por qué importa esto? Porque los fundamentos son, precisamente, los cimientos. Un estudiante que arrastra un hueco en, digamos, cómo funcionan las funciones o la recursión, va a tropezar una y otra vez más adelante, sin saber muy bien por qué. Sweigart, en su libro de Python para principiantes, dedica capítulos enteros a construir estos fundamentos paso a paso justamente porque son la base de todo lo que viene después [2].

Cuando la única señal que recibes es “pasó / no pasó”, terminas repitiendo lo que ya dominas (porque te da confianza) y evitando, sin darte cuenta, lo que no entiendes bien. El resultado es una falsa sensación de progreso.

1.3. La solución que propongo

La idea es un tutor con forma de videojuego (un RPG por niveles) que no se limite a decir “correcto”. En vez de eso, que **analice tu código** y te diga qué tan bien estás demostrando entender cada concepto. En concreto, el sistema:

1. Organiza el aprendizaje en “mundos” (un lenguaje cada uno) y “niveles” (un concepto cada uno), y a cada nivel le acompaña una lección pensada para alguien que parte de cero.
2. Usa un modelo de inteligencia artificial (un LLM) como cerebro del sistema: lee tu código y evalúa si *demuestra* comprensión, no solo si da el resultado correcto.
3. Lleva la cuenta, con un modelo probabilístico, de qué tan bien dominas cada concepto, y va ajustando esa estimación con cada intento.

4. Decide solo qué hacer contigo: avanzarte, hacerte repetir, generarte un ejercicio de refuerzo, o explicarte el concepto de otra forma si ves que no lo captaste.

2. Qué hace la IA en este proyecto

Según el alcance del Summer Camp, la IA tenía que ser el motor real de la solución, no un adorno. En este proyecto la IA cumple, de hecho, con **dos** de los resultados válidos al mismo tiempo:

- **Personalización:** el contenido y el ritmo se adaptan a cada persona. Si dominas algo, avanzas; si te cuesta, el sistema se detiene, te refuerza, o te explica el concepto de otra manera.
- **Automatización:** después de cada intento, el sistema decide solo qué sigue — sin que nadie tenga que revisarlo a mano — y hasta genera ejercicios nuevos cuando hace falta.

Vale la pena aclarar por qué esto no es “un chatbot pegado a un formulario”. La diferencia está en qué se hace con la respuesta de la IA: en vez de mostrarte un texto para que lo leas, el sistema le pide a la IA que analice tu código y devuelva un resultado *con estructura* (un JSON con puntajes), y ese resultado alimenta directamente a otro modelo que toma las decisiones. Es decir, la salida de la IA no es el producto final que ves — es un dato interno que mueve toda la maquinaria. Por eso decimos que la IA es el núcleo computacional, y no un extra.

3. Arquitectura de Implementación

3.1. Cómo fluyen los datos

Cada vez que envías una solución, pasa lo siguiente, en orden:

1. Escribes tu código para el ejercicio y le das a enviar.
2. El **sandbox** lo ejecuta de verdad — compila C con **gcc**, Java con **javac**, y corre las consultas SQL sobre una base de datos SQLite real — para ver si funciona.
3. El **LLM** (puede ser Groq, Anthropic o Gemini, se pueden intercambiar) lee tu código y clasifica qué tan bien dominas cada concepto, devolviéndolo como JSON.
4. El **modelo BKT** toma ese resultado y actualiza su estimación de cuánto dominas el concepto.
5. El **recomendador** mira esa estimación y decide qué sigue: avanzarte, hacerte repetir, generarte un ejercicio de refuerzo o reforzar el concepto más adelante.

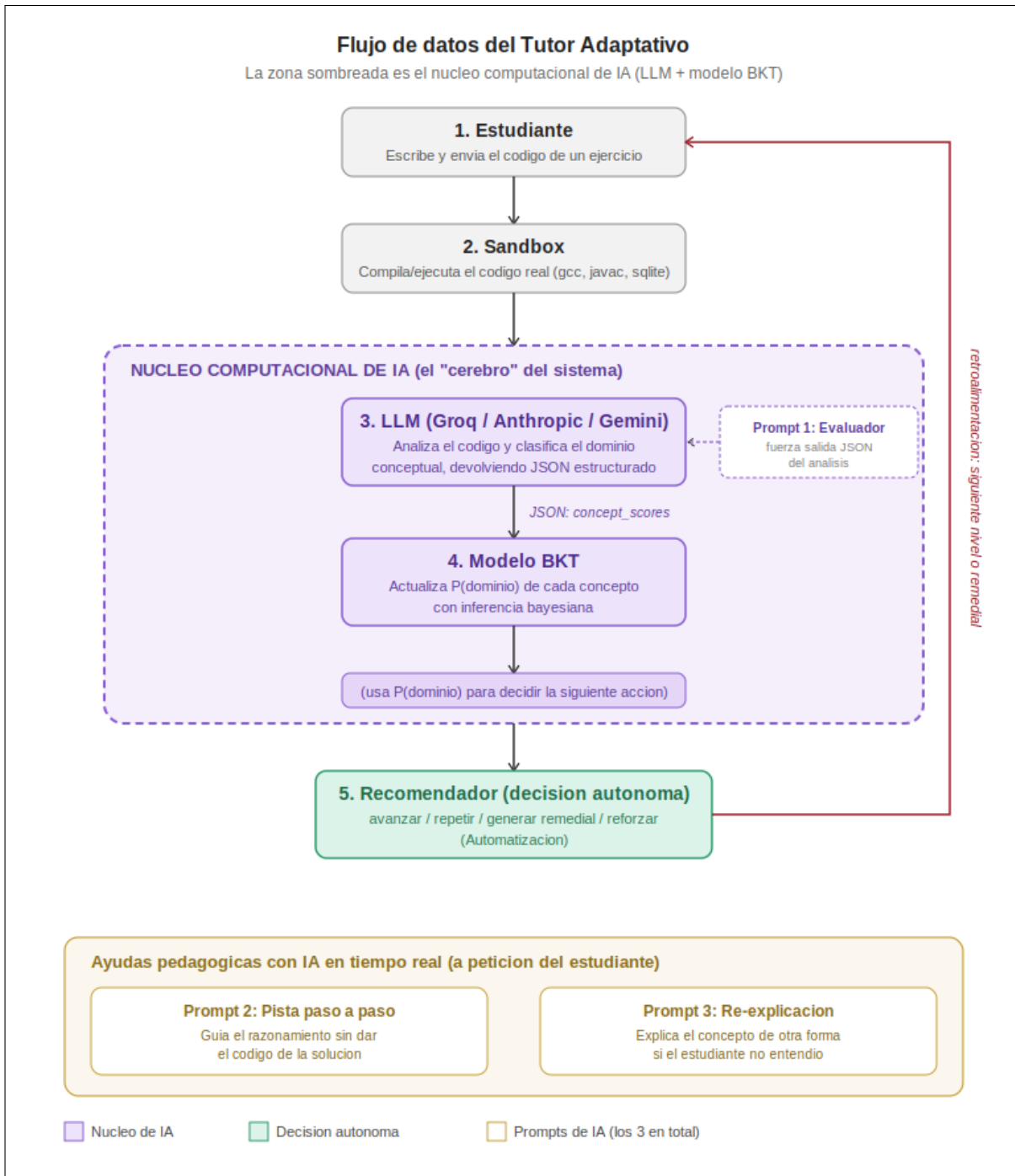


Figura 1: Arquitectura del motor de IA: flujo desde el envío del estudiante hasta la decisión del sistema.

3.2. Los cuatro lenguajes y sus referencias

Elegí cuatro lenguajes a propósito, porque cada uno representa una forma distinta de pensar la programación, y entre todos obligan al sistema a demostrar que funciona en escenarios muy diferentes:

- **Python** — dinámico y flexible, ideal para empezar. La currícula de este mundo se apoya en el enfoque para principiantes de Sweigart [2], que construye los fundamentos sin dar nada por sentado.
- **C** — de bajo nivel, con manejo manual de memoria y punteros. Aquí la referencia es el clásico de Kernighan y Ritchie [1], escrito por los propios creadores del lenguaje.
- **Java** — orientado a objetos y de tipado estático. Para este mundo tomé como guía el enfoque didáctico de *Head First Java* [3], pensado para que los conceptos de POO “entren” de verdad.
- **SQL** — declarativo, distinto a todos los demás: no describes *cómo* hacer algo, sino *qué* quieres obtener. La referencia es el libro de Beaulieu [4], un estándar para aprender consultas desde cero.

3.3. Componentes técnicos

Módulo	Responsabilidad
content/*.json	Currícula: teoría, lecciones, ejercicios, tests y soluciones (32 niveles).
sandbox.py	Ejecuta y verifica la solución del estudiante, con una estrategia por lenguaje.
llm_judge.py	Núcleo de IA. Llama al LLM para clasificar el dominio conceptual en JSON.
mastery.py	Modelo BKT: mantiene P(dominio) por concepto.
recommender.py	Motor de decisión sobre la siguiente acción.
exercise_generator.py	Genera ejercicios remediales con IA.
app.py	Interfaz visual (Streamlit).

Cuadro 1: Módulos principales del sistema.

3.4. Herramientas y tecnologías utilizadas

- **Lenguaje base:** Python 3.
- **Interfaz:** Streamlit.
- **Modelo de dominio:** Bayesian Knowledge Tracing (BKT), implementación propia.
- **Proveedores de LLM (intercambiables):** Groq, Anthropic y Google AI Studio (Gemini).
- **Ejecución de código:** gcc (C), javac/java (Java), sqlite3 (SQL), intérprete de Python.
- **Control de versiones:** Git y GitHub.

4. Sistema de Aprendizaje Progresivo

Más allá de evaluar, el sistema está diseñado para **enseñar** a alguien sin conocimiento previo. Cada nivel ofrece tres niveles de ayuda, de menor a mayor grado de asistencia, para que el estudiante recurra a cada uno solo cuando lo necesite. Dos de estas tres ayudas usan la IA en tiempo real, reforzando el criterio de **Personalización**.

4.1. Ayuda 1: Lección completa (contenido estático)

Cada uno de los 32 niveles incluye una lección escrita para principiantes totales: una frase-resumen inicial (“En una frase: ...”), una explicación con analogía de la vida real, y un ejemplo resuelto *distinto* al ejercicio a resolver (para no revelar la respuesta).

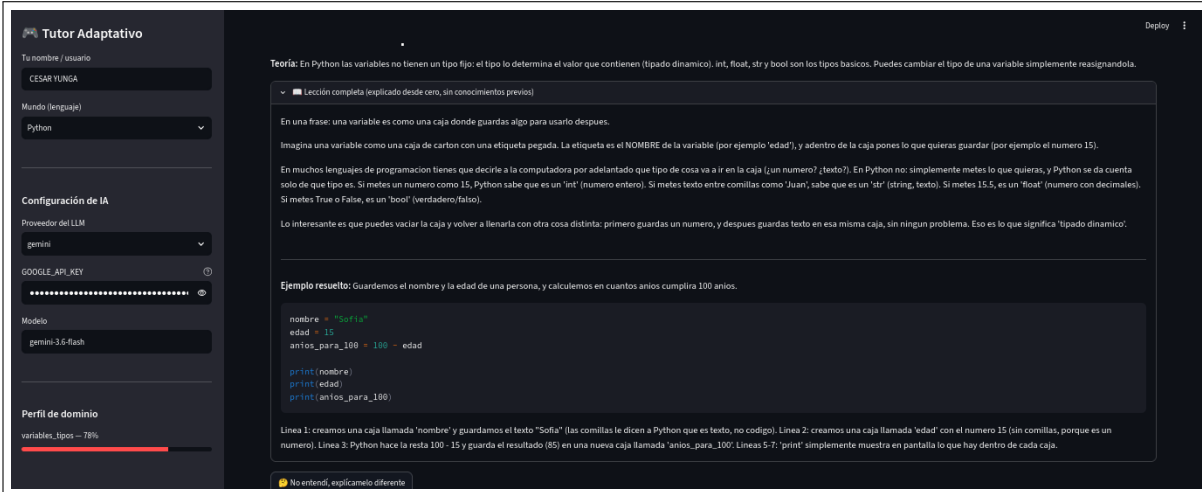


Figura 2: Lección completa desplegada: frase-resumen (“En una frase...”), explicación con analogía y ejemplo resuelto, todo para principiantes sin conocimiento previo.

4.2. Ayuda 2: Pista paso a paso (IA en tiempo real)

Si el estudiante está atascado, un botón solicita a la IA una pista de razonamiento — que guía sin revelar el código. El prompt de sistema que fuerza este comportamiento es:

```

1 HINT_SYSTEM_PROMPT = """Eres un tutor de programacion ayudando a un
2 estudiante principiante que esta atascado en un ejercicio. Te daran el
3 enunciado del ejercicio y el codigo base (sin resolver).
4
5 Tu tarea es dar una PISTA en pasos de razonamiento -- NUNCA escribas el
6 codigo completo de la solucion ni la linea exacta que hay que escribir.
7 Divide el problema en 3 a 5 pasos de pensamiento (que revisar primero,
8 que estructura de control conviene usar, que caso especial no hay que
9 olvidar), en lenguaje simple para alguien sin experiencia previa.
10
11 Responde en espanol, en texto plano, con los pasos numerados, sin
12 codigo completo."""

```

Listing 1: Prompt de la pista (src/llm_judge.py)

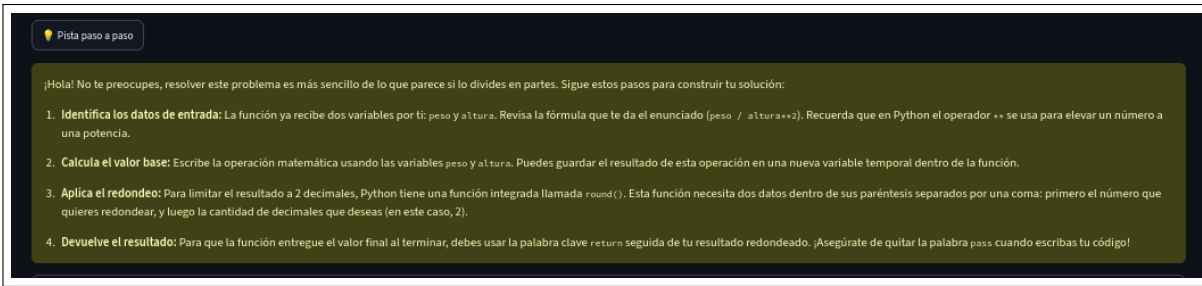


Figura 3: Ejecución real del prompt de la pista: la IA (Gemini) genera pasos de razonamiento para el ejercicio, sin escribir el código de la solución.

4.3. Ayuda 3: Solución paso a paso (solucionario detallado)

Como último recurso, el estudiante puede abrir la solución completa del ejercicio, explicada **símbolo por símbolo** (qué hace cada `&`, cada `;`, cada comilla, cada `return`). Está pensada para depurar el propio intento cuando falla por un detalle mínimo de sintaxis, no para copiar.

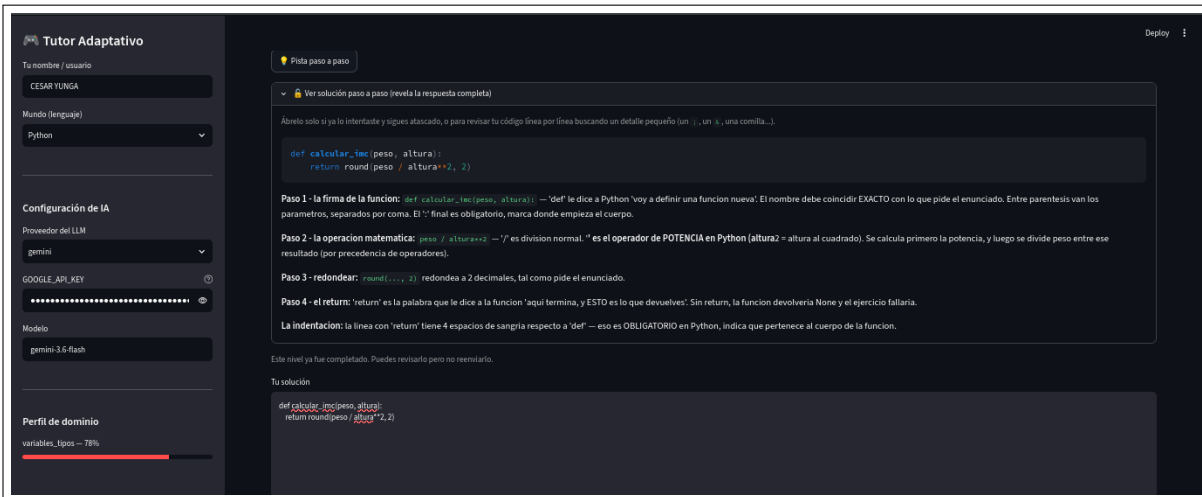


Figura 4: Solución paso a paso desplegada: el código de la solución seguido de la explicación detallada, símbolo por símbolo.

4.4. Re-explicación adaptativa (IA en tiempo real)

Adicionalmente, un botón permite pedirle a la IA que re-explique el concepto de una forma *completamente distinta* si la explicación original no se entendió. El prompt correspondiente:

```
1 REEXPLAIN_SYSTEM_PROMPT = """Eres un profesor paciente que explica
2 fundamentos de programacion a alguien que nunca ha programado antes.
3 Se te dara un concepto y su explicacion actual, que la persona NO
4 entendio. Tu trabajo es explicarlo de nuevo, de una forma COMPLETAMENTE
5 DISTINTA a la explicacion original -- usa una analogia diferente, un
6 ejemplo de la vida cotidiana distinto, y un lenguaje todavia mas
7 simple. Maximo 3 parrafos cortos. Responde en espanol, en texto plano.
  """
```

Listing 2: Prompt de re-explicación (src/llm_judge.py)

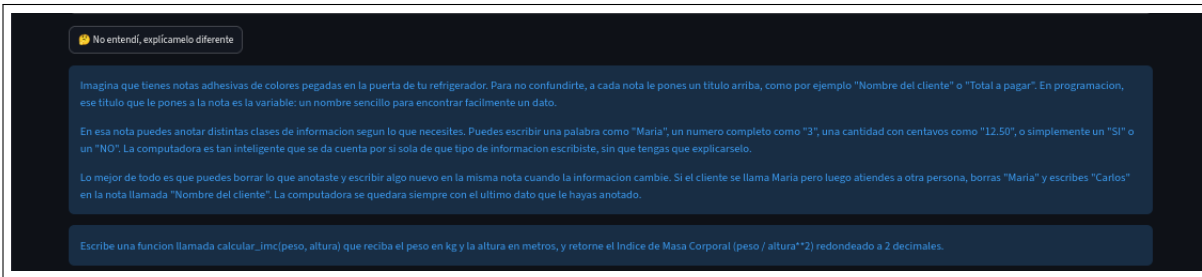


Figura 5: Ejecución real del prompt de re-explicación: la IA explica el mismo concepto con una analogía completamente distinta (notas adhesivas en un refrigerador).

Este sistema de tres ayudas fue una decisión de diseño posterior a la versión inicial: tras probar el prototipo, se identificó que un tutor que solo evalúa no sirve para *aprender* desde cero, así que se añadió contenido pedagógico progresivo y dos usos adicionales de IA en tiempo real (pista y re-explicación).

5. Evidencia de Ejecución

5.1. El programa en ejecución

La siguiente captura muestra la aplicación arrancando, con el mapa de niveles de un mundo (lenguaje) y la barra lateral de configuración.

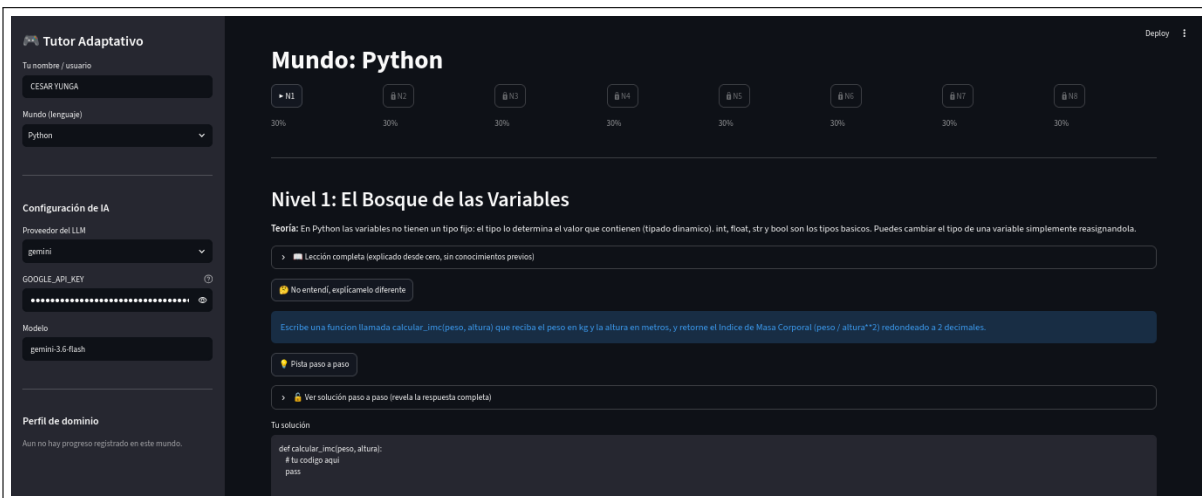


Figura 6: La aplicación en ejecución: mapa de niveles del mundo seleccionado y barra lateral de configuración (proveedor de IA, modelo, usuario).



Figura 7: Vista inicial de un mundo (C): la teoría del nivel, la lección desplegable, los botones de ayuda y el enunciado del ejercicio, tal como los ve el estudiante al empezar.

5.2. Verificación de los 4 lenguajes con análisis real de IA

Cada uno de los cuatro lenguajes fue probado enviando una solución y confirmando dos cosas: que los tests pasan (“Tests superados: 100 %”) y que la retroalimentación proviene de la IA en tiempo real (etiqueta “fuente: llm”). Esto verifica de extremo a extremo tanto el **sandbox** (que compila/ejecuta el código real) como el **núcleo de IA**.

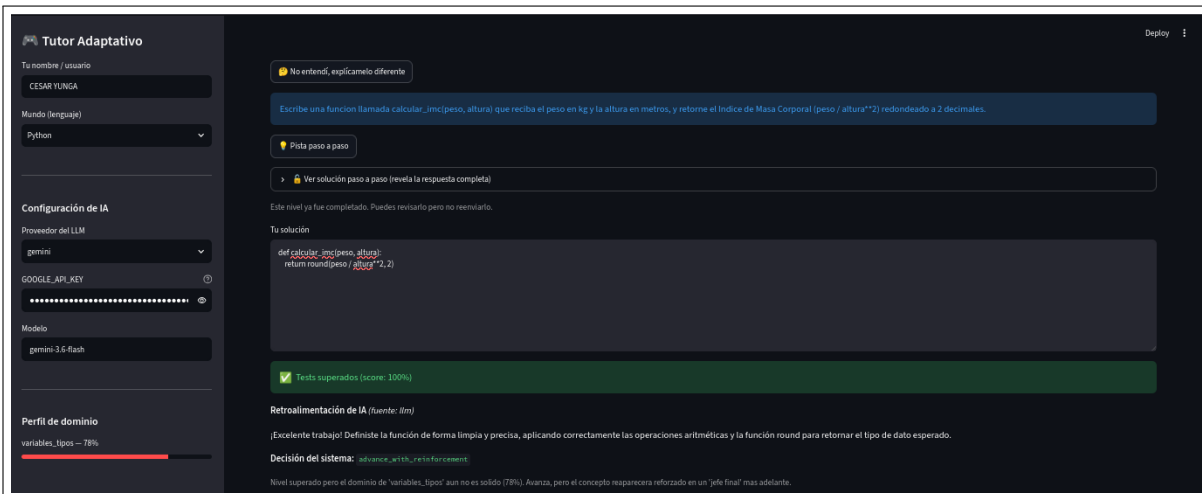


Figura 8: PYTHON: ejercicio resuelto, tests superados y retroalimentación con “fuente: llm”.

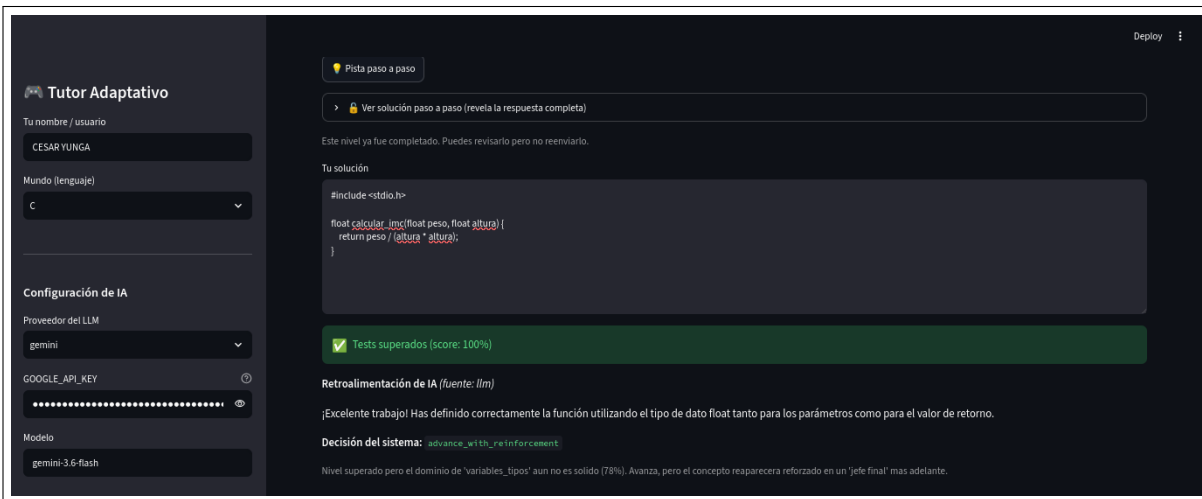


Figura 9: C: ejercicio resuelto y verificado (compilado con gcc real), con retroalimentación de la IA.

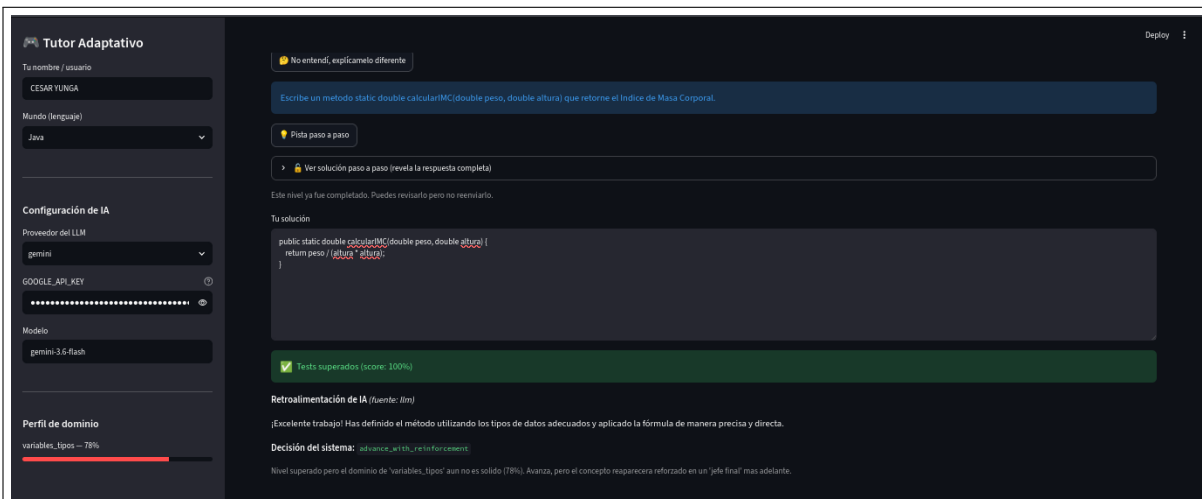


Figura 10: JAVA: ejercicio resuelto y verificado (compilado con javac real), con retroalimentación de la IA.

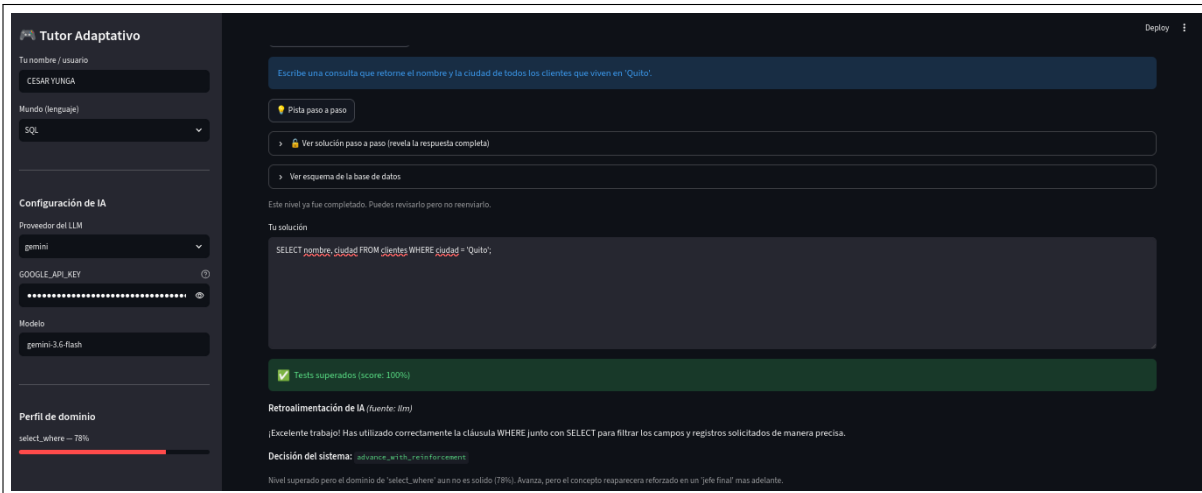


Figura 11: SQL: consulta resuelta y verificada (ejecutada sobre SQLite real), con retroalimentación de la IA.

5.3. El prompt de sistema (núcleo de IA)

El siguiente es el mensaje fijo que se envía al LLM antes del código del estudiante. Obliga al modelo a responder únicamente en formato JSON estructurado, lo que permite integrar su salida directamente en la lógica del programa (a diferencia de un chatbot que respondería en texto libre).

```

1 JUDGE_SYSTEM_PROMPT = """Eres un evaluador tecnico de codigo para
2 una plataforma educativa de fundamentos de programacion. Recibiras:
3 el enunciado de un ejercicio, los conceptos que evalua, y el codigo
4 que un estudiante escribio como solucion.
5
6 Tu tarea es analizar el CODIGO EN SI (no solo si el resultado es
7 correcto) y devolver UNICAMENTE un JSON valido con esta forma exacta:
8
9 {
10     "concept_scores": {"<concepto>": <float 0.0-1.0>},
11     "detected_issues": ["<descripcion de un error detectado>"],
12     "feedback": "<1-2 frases de retroalimentacion, en espanol>"
13 }
14 """

```

Listing 3: Prompt de sistema del evaluador (src/llm_judge.py)

6. Uso de la IA como Copiloto (Trazabilidad)

Conforme a los requisitos de documentación del Summer Camp, se registra el uso de IA como copiloto durante el desarrollo.

6.1. Distinción importante

Deben distinguirse dos usos de IA en este proyecto:

1. **IA como copiloto de desarrollo:** la herramienta que ayudó a *construir* el proyecto (esta sección). Es análoga a usar un editor de código o un compilador.

2. **IA en tiempo de ejecución:** la que la *propia aplicación* usa cada vez que un estudiante resuelve un ejercicio (Groq/Gemini/Anthropic). Esta es el núcleo computacional exigido por la rúbrica.

6.2. Herramienta de copiloto utilizada

Herramienta	Uso
Claude (Anthropic)	Diseño de arquitectura, generación del esqueleto de código (motor BKT, sandbox, integración LLM, recomendador), currícula de los 32 niveles, depuración de la conexión con proveedores de IA, y apoyo en la configuración de Git/-GitHub.
OpenAI Tokenizer	Herramienta de análisis de tokens vista en clase; usada como apoyo conceptual para entender el consumo de tokens y el comportamiento de los modelos “razonadores”.

Cuadro 2: Herramientas de IA usadas como copiloto de desarrollo.

6.3. Modelos usados en tiempo de ejecución

El sistema soporta tres proveedores intercambiables. Durante las pruebas se verificó su funcionamiento con Google AI Studio (Gemini, modelo `gemini-3.6-flash`), confirmando análisis conceptual real y específico sobre el código del estudiante.

6.4. Nota sobre depuración de proveedores

Durante el desarrollo se probaron los tres proveedores con resultados distintos, documentados como parte del proceso de ingeniería:

- **Groq:** la conexión externa funcionaba (verificado con `curl`), pero fallaba de forma intermitente dentro de la aplicación. Se identificó que el modelo `gpt-oss-120b` es un modelo “razonador” que consume parte del límite de tokens en razonamiento interno; se ajustó `max_tokens` en consecuencia.
- **Gemini:** funcionando y verificado *end-to-end*.

El diseño con proveedor intercambiable permitió que un fallo de conectividad con un proveedor no bloqueara el proyecto.

7. Cómo Ejecutar el Proyecto

El proyecto es multiplataforma (Windows, macOS o Linux). No requiere una máquina virtual — esta se usó únicamente en el entorno de desarrollo del autor.

7.1. Requisitos

- Python 3 instalado.
- Una API key de un proveedor de IA (Groq, Anthropic o Google AI Studio; todos tienen capa gratuita).
- Para el mundo C: el compilador `gcc`. Para el mundo Java: el JDK completo (`javac`). Python y SQL no requieren nada adicional.

7.2. Instalación y ejecución

```
1 # 1. Clonar el repositorio
2 git clone https://github.com/Kaiman-p/tutor-adaptativo-ia.git
3 cd tutor-adaptativo-ia
4
5 # 2. Instalar dependencias
6 pip install -r requirements.txt
7
8 # 3. Ejecutar la aplicacion
9 streamlit run src/app.py
```

Listing 4: Pasos para ejecutar el proyecto

La aplicación se abre en el navegador (`localhost:8501`). En la barra lateral se configura el proveedor de IA, el modelo y la API key. Sin API key, el sistema funciona en modo *fallback* (solo verificación de tests, sin análisis conceptual de IA).

8. Resultados y Verificación

Los 32 niveles (4 lenguajes $\times$ 8 niveles) fueron verificados con ejecución real, no solo revisados manualmente:

- **Python, C, SQL:** las soluciones de referencia se probaron contra los tests reales de cada nivel mediante un script automatizado.
- **Java:** compilado y ejecutado con `javac` y `java` reales en la máquina del autor.
- **Interfaz completa:** probada con el framework oficial de testing de Streamlit (`AppTest`), simulando interacciones reales.

9. Conclusiones

El proyecto demuestra que la IA puede ser el núcleo computacional real de una herramienta educativa, no un componente decorativo. La combinación de un LLM (para análisis conceptual del código) con un modelo probabilístico clásico (BKT, para modelar el dominio) permite tomar decisiones pedagógicas autónomas y personalizadas, cumpliendo simultáneamente los resultados de Personalización y Automatización definidos en el alcance del Summer Camp.

El diseño modular y con proveedor de IA intercambiable demostró su valor durante el desarrollo, al permitir sortear problemas de conectividad sin rediseñar el sistema.

Declaración de uso de Inteligencia Artificial

Este proyecto fue desarrollado con el apoyo de **Claude (Anthropic)** como herramienta de copiloto de programación, conforme a los lineamientos del Summer Camp que permiten y exigen documentar el uso de IA. El autor revisó, probó y comprende cada componente del sistema, y tomó las decisiones de diseño del proyecto (selección de lenguajes, elección del modelo de dominio,

proveedores de IA, y enfoque pedagógico). El detalle completo de las sesiones de trabajo con IA se encuentra en el archivo `docs/registro_prompts.md` del repositorio.

Referencias

- [1] Kernighan, B. W. & Ritchie, D. M. (1988). *The C Programming Language* (2^a ed.). Prentice Hall. ISBN 978-0131103627. Disponible en: [https://seriouscomputerist.atariverse.com/media/pdf/book/C%20Programming%20Language%20-%202nd%20Edition%20\(OCR\).pdf](https://seriouscomputerist.atariverse.com/media/pdf/book/C%20Programming%20Language%20-%202nd%20Edition%20(OCR).pdf).
- [2] Sweigart, A. (2020). *Automate the Boring Stuff with Python* (2^a ed.). No Starch Press. Disponible en línea: <https://automatetheboringstuff.com/>.
- [3] Sierra, K. & Bates, B. (2005). *Head First Java* (2^a ed.). O'Reilly Media. ISBN 978-0596009205. Disponible en: https://archive.org/details/HeadFirstJava2ndEdition_201511.
- [4] Beaulieu, A. (2020). *Learning SQL* (3^a ed.). O'Reilly Media. ISBN 978-1492057611. Disponible en: https://archive.org/details/learningsql0000beau_s2h5.